Action-Prefix System — Deep Web Research

Field	Value
Revision	1
Created	2026-06-09
Last modified	2026-06-09T00:00:00Z
Status	active
Scope	Universal "ACTION_NAME ::" prompt-prefix system for the Helix Constitution (new anchor §11.4.140)
Authority	research input for `DESIGN.md` + `RULE_DRAFT.md`

> Anti-bluff (§11.4.6): every claim below cites at least one source URL. Where a mechanism could NOT be confirmed for an agent, it is marked `UNCONFIRMED:` or an honest SKIP per §11.4.3. No mechanism is asserted as fact without a cited primary source.

. Research question

Design a UNIVERSAL, extensible prompt-prefix system in which a user prompt that STARTS with `ACTION_NAME ::` (e.g. `BACKGROUND :: IMPORTANT: do X`) causes the agent to (a) replace the `ACTION_NAME ::` prefix with that action's registered

expansion text, then (b) execute the rest of the prompt. It must work with EVERY CLI coding agent (not just Claude Code), be fully decoupled + reusable by every project that includes the constitution submodule, and load + execute out of the box.

Two sub-questions drive the research:

1. **What prompt-interception / preprocessing mechanism does each major CLI agent expose?** (the LAYER-2 mechanical seam)
2. **What is the universal always-on instruction-context carrier each agent reads?** (the LAYER-1 self-applied seam — the floor that works even when no mechanical hook exists)

[1]

---

[1] [1] . Per-agent prompt-interception + instruction-context matrix

. Claude Code (Anthropic)

LAYER 2 (mechanical) — STRONG. Two relevant hooks.

Claude Code exposes a `UserPromptSubmit` hook that "fires before the agent sees your prompt and the hook can rewrite the prompt, append context, or block submission … runs right after you hit Enter and before Claude sees the prompt" (egghead.io UserPromptSubmit lesson; Claude Code Hooks guide).

The official hooks guide confirms the lifecycle event table includes BOTH:

- `UserPromptSubmit` — "When you submit a prompt, before Claude processes it."
- `UserPromptExpansion` — "When a user-typed command expands into a prompt, before it reaches Claude. Can block the expansion."

(Claude Code Hooks guide event table)

Hook I/O contract (cited, verbatim from the guide):

- The hook is a `"type": "command"` shell command configured under a `hooks` block in a settings file (`settings.json` / `.claude/settings.json` / `.claude/settings.local.json`).

- The command receives the event JSON on **stdin**. For `UserPromptSubmit` the payload carries the `prompt` text (per the guide: "`UserPromptSubmit` hooks get the `prompt` text").
- **Exit-code semantics:** "For `UserPromptSubmit`, `UserPromptExpansion`, and `SessionStart` hooks, anything you write to **stdout** is added to Claude's context" (exit 0). Exit 2 blocks the prompt and feeds stderr back to Claude.
- **Structured output:** "For `UserPromptSubmit` hooks, use `additionalContext` instead to inject text into Claude's context" — i.e. `{"hookSpecificOutput": {"hookEventName": "UserPromptSubmit", "additionalContext": "…"}}`.

(Claude Code Hooks guide — Hook output / Structured JSON output sections)

Important nuance for our design (honest, per §11.4.6): the *documented, guaranteed-stable* contract for `UserPromptSubmit` is **additive** — stdout / `additionalContext` is *added to* Claude's context, not a guaranteed *replacement* of the literal prompt text. Some community write-ups describe "rewriting"/"transforming" the prompt (e.g. egghead, Developers Digest UserPromptSubmit guide), and the guide itself warns "Heavy transformations confuse users when the model responds to something they didn't type — keep transforms visible." The robust, spec-guaranteed behaviour we can rely on is: **the hook detects the `ACTION_NAME ::` prefix and injects the registered expansion as `additionalContext`** (an instruction the model then obeys), rather than silently mutating the prompt string. This is the same end-user effect (the expansion is applied) with a contract the docs guarantee.

- **LAYER 1 carrier:** `CLAUDE.md` (+ `.claude/rules/*.md`) — the `InstructionsLoaded` event in the guide confirms "When a CLAUDE.md or `.claude/rules/*.md` file is loaded into context." So even with NO hook, the recognition instruction embedded in `CLAUDE.md` is always loaded.
- **Skills / plugins / MCP** are additional Layer-2 surfaces but are task-triggered, not prompt-prefix-triggered, so they are secondary here.

> **Verdict — Claude Code: LAYER 1 (CLAUDE.md) + LAYER 2 (UserPromptSubmit / UserPromptExpansion hook). Mechanical interception available.**

. Gemini CLI (Google)

LAYER 2 (mechanical) — PARTIAL. Gemini CLI's nearest analogue to a prompt-rewrite seam is **custom commands** (TOML files under `~/.gemini/commands/` or `<project>/.gemini/commands/`). A command file has a `prompt` field, and "If your prompt contains the special placeholder `{{args}}`, the CLI will replace that placeholder with the text the user typed after the command name." File-content injection (`@{...}`) and shell injection (`!{...}`) are processed before argument substitution (Gemini CLI custom commands; Google Cloud blog — custom slash commands).

This is a **slash-command** router, not a generic `^PREFIX ::` interceptor — it fires on `/command`, not on an arbitrary anchored uppercase prefix. So Gemini's mechanical seam can host an *equivalent* (e.g. a `/background` command whose `{{args}}` expansion equals the BACKGROUND text), but it cannot transparently rewrite a free-form `BACKGROUND :: …` line. `UNCONFIRMED:` whether Gemini CLI exposes a pre-submit hook equivalent to Claude's `UserPromptSubmit` — the custom-commands + GEMINI.md docs do not describe one.

- **LAYER 1 carrier:** `GEMINI.md` context files — "loads various context files from several locations, concatenates the contents of all found files, and sends them to the model with every prompt" (Gemini CLI GEMINI.md docs). This is the always-on seam: the recognition instruction in `GEMINI.md` is sent with every prompt.

> **Verdict — Gemini CLI: LAYER 1 (GEMINI.md) primary + LAYER 2 (custom slash-command equivalent, NOT transparent prefix-rewrite).**[3]

. Qwen Code (Alibaba / QwenLM)

Qwen Code "is forked from Gemini CLI … adapted specifically for use with the Qwen3-Coder model" and "Both tools share Gemini CLI's custom slash commands and shell integration." The only structural difference is the config directory

(`.qwen/`) and context file (`QWEN.md` , configurable via `context.fileName`) (Qwen Code review — elite-ai-assisted-coding; Qwen Code configuration — zdoc; Qwen Code repo).

- **LAYER 1 carrier:** `QWEN.md` (same concatenate-and-send semantics as Gemini's `GEMINI.md`).
- **LAYER 2:** same TOML custom-command slash-command equivalent as Gemini.

> **Verdict — Qwen Code: identical to Gemini CLI (LAYER 1 QWEN.md + LAYER 2 custom slash-command equivalent).**[4]

. OpenAI Codex CLI

LAYER 1 carrier — STRONG, standard. Codex reads `AGENTS.md` (and `AGENTS.override.md`) walking from the project root down to the cwd, concatenating, with closer files overriding (Codex AGENTS.md guide; Codex config reference).

LAYER 2 — PARTIAL. Codex has **custom prompts** stored in a `prompts/` directory, surfaced via the `/prompts:` slash menu (Codex CLI features) and `config.toml` under `~/.codex/` . As with Gemini, this is a slash-command-style surface, not a transparent free-form `^PREFIX ::` interceptor. `UNCONFIRMED:` whether Codex exposes a pre-submit prompt-rewrite hook.

> **Verdict — Codex CLI: LAYER 1 (AGENTS.md) primary + LAYER 2 (custom-prompt slash equivalent).**[5]

. GitHub Copilot CLI

LAYER 1 carrier — STRONG, standard. Copilot CLI reads `.github/copilot-instructions.md` at repo root AND `AGENTS.md` files (root, cwd, or paths in `COPILOT_CUSTOM_INSTRUCTIONS_DIRS`), plus path-specific `*.instructions.md`

under `.github/instructions/`. "Instructions are automatically added to requests that you submit to Copilot" (GitHub Docs — add custom instructions for Copilot CLI; GitHub Changelog — Copilot coding agent AGENTS.md).

LAYER 2 — PARTIAL. Custom agents + the Copilot CLI skill tool exist (GitHub Docs — create custom agents for CLI), but `UNCONFIRMED:` whether any transparent pre-submit prompt-rewrite hook exists.

> **Verdict — Copilot CLI: LAYER 1 (AGENTS.md + copilot-instructions.md) primary + LAYER 2 (custom-agent / skill equivalent).**[6]

. Cursor

LAYER 1 carrier — STRONG. Cursor reads `.cursor/rules/*.mdc` rule files; a rule with `alwaysApply: true` "loads for every single conversation regardless of context" — it is converted into a system prompt for the agent (Cursor Rules docs; Vibe Coding Academy — Cursor Rules guide). Cursor is also a documented `AGENTS.md` consumer (agents.md).

LAYER 2 — none documented as a transparent free-form prefix interceptor.

> **Verdict — Cursor: LAYER 1 (.cursor/rules `alwaysApply: true`, and/or AGENTS.md) only.** Mechanical free-form interception not available → honest SKIP of Layer 2; Layer 1 fully covers it.[7]

. Aider

LAYER 1 carrier — present. Aider loads a conventions file via `/read CONVENTIONS.md` or `aider --read CONVENTIONS.md`, or persistently via `read: CONVENTIONS.md` in `.aider.conf.yml`; "aider will forward these conventions to the currently used LLM" (Aider — specifying coding conventions; Aider YAML config). Aider is also a documented `AGENTS.md` consumer (agents.md).

LAYER 2 — none documented as a transparent prefix interceptor.

> **Verdict — Aider: LAYER 1 (CONVENTIONS.md via `.aider.conf.yml read:`, and/or AGENTS.md) only.**

. Cline / Continue / Roo Code

LAYER 1 carrier — present. Cline reads `.clinerules/*.md` (merged) and the cross-tool global `~/.agents/AGENTS.md`; AGENTS.md root support is requested/tracked (Cline Rules docs; cline/cline#5033 AGENTS.md). Continue reads `.continue/rules/` (Markdown/YAML) and tracks AGENTS.md support (continuedev/continue#6716). Roo Code has custom-instructions files (Roo Code custom instructions).

LAYER 2 — none documented as a transparent prefix interceptor.

> **Verdict — Cline/Continue/Roo: LAYER 1 (.clinerules / .continue/rules / AGENTS.md) only.**

. AGENTS.md — the universal LAYER-carrier

`AGENTS.md` is "a simple, open format for guiding coding agents, used by over 60k open-source projects," that "emerged from collaborative efforts across … OpenAI Codex, Amp, Jules from Google, Cursor, and Factory" and is now "stewarded by the Agentic AI Foundation under the Linux Foundation." It "works across tools like Cursor, Windsurf, Aider, and Jules," with documented support from GitHub Copilot, OpenAI Codex, Google, Amp, Factory, RooCode, Zed, and Warp (agents.md; InfoQ — AGENTS.md open standard; agentsmd/agents.md repo).

Notably: "Claude Code has not yet adopted AGENTS.md" as a first-class native file (agents.md adoption note), which is exactly why the Helix Constitution maintains BOTH `CLAUDE.md` (Claude Code's native carrier) AND `AGENTS.md` / `QWEN.md` / `GEMINI.md` — the established multi-file propagation pattern (§11.4.35) already solves the "different agent reads a different file" problem.

Consequence for our design: the LAYER-1 recognition instruction must be mirrored into every carrier the constitution already maintains (`CLAUDE.md` + `AGENTS.md` + `QWEN.md` + a `GEMINI.md` mirror). That set already covers every agent above as the floor.
[3]

. Open-source building blocks for prompt-prefix / macro / router systems

The closest reusable prior art:

- **Slash-command routers** are the de-facto pattern every agent ships (Claude Code skills/commands, Gemini/Qwen TOML custom commands, Codex `prompts/` , Copilot custom agents). They map a *literal command token → expansion text with* `{{args}}` *substitution* (Gemini custom commands; Codex features). Our action registry is the same model with the trigger generalised from `/command` to `^ACTION_NAME ::` .
- **Macro-expansion / preprocessor model** (the C preprocessor mental model): a *directive* at line start is *replaced by its registered definition*, then the result is *rescanned* and executed. ppstep documents the canonical expand→rescan steps (ppstep; GNU cpp macro expansion). Our `ACTION_NAME ::` prefix is a single-token, line-anchored macro: detect → substitute the registered expansion → execute the remainder. This is the authoritative semantic precedent for "replace prefix with expansion then run the rest."
- **Prompt-template engines** (OpenPrompt; embedded-directive template engines) establish the registry-of-named-templates + variable-substitution pattern we reuse for the registry data file (OpenPrompt).

- **UserPromptSubmit hook write-ups** establish the concrete "detect a suffix/prefix token and rewrite/expand it before the model sees it" recipe on Claude Code (e.g. `:plan` → "Create a detailed step-by-step plan for: {task}") (egghead UserPromptSubmit; Developers Digest).

Conclusion: there is NO single off-the-shelf cross-agent "prefix-action" package. The reusable building blocks are (a) the slash-command/registry model (every agent), (b) the C-preprocessor expand-then-rescan semantics (mental model + grammar), and (c) Claude Code's `UserPromptSubmit` / `UserPromptExpansion` hooks (the one agent with a robust mechanical pre-submit seam). The Helix system is therefore an ORIGINAL composition of these blocks (cite per §11.4.8: "NO single external prefix-action package found — original composition of the slash-command registry model + C-preprocessor expand-then-rescan semantics + Claude Code UserPromptSubmit hook").

. Per-agent integration-mechanism summary table

Agent	LAYER 1 carrier (always-on, self-applied)	LAYER 2 mechanical pre-submit seam	Transparent `^PREFIX::` interception?
Claude Code	`CLAUDE.md` + `.claude/rules/*.md`	`UserPromptSubmit` / `UserPromptExpansion` hook (stdin JSON, `additionalContext`)	**YES (hook)**
Gemini CLI	`GEMINI.md` (concat + sent every prompt)	TOML custom command (`{{args}}`), slash-triggered	NO (slash equivalent only)
Qwen Code	`QWEN.md` (Gemini fork)	TOML custom command (`{{args}}`), slash-triggered	NO (slash equivalent only)
OpenAI Codex CLI	`AGENTS.md` / `AGENTS.override.md`	`prompts/` custom prompts (`/prompts:`), slash-triggered	NO (slash equivalent only)
GitHub Copilot CLI	`AGENTS.md` + `.github/copilot-instructions.md`	custom agents / skill tool	NO
Cursor	`.cursor/rules/*.mdc` (`alwaysApply:true`) + AGENTS.md	—	NO (Layer 1 only)
Aider	`CONVENTIONS.md` (`.aider.conf.yml read:`) + AGENTS.md	—	NO (Layer 1 only)
Cline	`.clinerules/*.md` + `~/.agents/AGENTS.md`	—	NO (Layer 1 only)
Continue	`.continue/rules/` + AGENTS.md (tracked)	—	NO (Layer 1 only)

Agent	LAYER 1 carrier (always-on, self-applied)	LAYER 2 mechanical pre-submit seam	Transparent `^PREFIX ::` interception?
Roo Code	custom-instructions files + AGENTS.md	—	NO (Layer 1 only)

Honest §11.4.3 SKIP statement: transparent, mechanical free-form `^ACTION_NAME ::` *string interception before the model sees it* is genuinely available ONLY on Claude Code (via `UserPromptSubmit` / `UserPromptExpansion`). For every other agent the equivalent is either (a) a slash-command the user must type instead of the free-form prefix, OR (b) LAYER 1 only — the agent itself recognises the prefix because the constitution's instruction-context file told it to. LAYER 1 is therefore the universal floor that makes the system work out-of-the-box on EVERY agent; LAYER 2 is the mechanical upgrade where the agent exposes a pre-submit seam.[5]

. Key reusable conclusion for DESIGN.md

1. **Two-layer architecture is correct and necessary.** No single mechanism is universal; the only universal seam is the instruction-context file every agent reads (LAYER 1). The mechanical hook (LAYER 2) is a strict, agent-specific upgrade.
2. **One shared data registry** (a tracked `actions/registry.yaml`) is the single source of truth both layers read — the LAYER-1 instruction tells the agent to consult it; the LAYER-2 hook parses it deterministically. This keeps the system decoupled (§11.4.28) and extensible (add a registry row → new action, no code change in either layer).
3. **The grammar is a line-anchored single-token macro** (C-preprocessor precedent): `^([A-Z][A-Z0-9_]*) ::` → replace with registered expansion → execute the remainder. Multiple stacked prefixes and a literal-escape rule are required (Section grammar in DESIGN.md).

4. `BACKGROUND ::` **is the first registry entry**, with the operator's verbatim expansion text.

Sources

- Claude Code Hooks guide
- egghead — Rewrite Prompts with UserPromptSubmit Hooks
- Developers Digest — UserPromptSubmit Hook
- Gemini CLI — Custom commands
- Gemini CLI — GEMINI.md context files
- Google Cloud blog — Gemini CLI custom slash commands
- Qwen Code repo
- Qwen Code configuration — zdoc
- Qwen Code review — elite-ai-assisted-coding
- Codex — AGENTS.md guide
- Codex — CLI features
- Codex — config reference
- GitHub Docs — Copilot CLI custom instructions
- GitHub Changelog — Copilot coding agent AGENTS.md
- Cursor — Rules docs
- Vibe Coding Academy — Cursor Rules guide
- Aider — Specifying coding conventions
- Aider — YAML config
- Cline — Rules docs
- cline/cline#5033 — AGENTS.md support
- continuedev/continue#6716 — AGENTS.md support
- Roo Code — custom instructions
- agents.md — open standard
- InfoQ — AGENTS.md open standard
- agentsmd/agents.md repo

- ppstep — preprocessor macro debugger
- GNU cpp — Macro Expansion internals
- OpenPrompt — open-source prompt-learning framework